LANL Threat Prediction Pipeline — Project 2 Write-Up

Team: Mark Crisci **Live URL:** https://lanlthreat.streamlit.app **Repo:** https://github.com/mcrisci24/lanl-threat-pipeline

1. Prediction question

*Given how a computer behaved in the **current** one-hour window, will the same computer show **red-team (attacker) activity in the next one-hour window**?* The "next window" framing is deliberate. Predicting the current window lets the model read features built from the very labels it is trying to predict — it scores near-perfect in evaluation and fails completely in production. The target column `target_redteam_next_window` is built with a Spark window function that shifts the red-team flag forward by one hour per computer, every current-window red-team aggregate (`redteam_event_count`, `redteam_current_flag`, etc.) is dropped from the feature set, and the training script asserts at runtime that no leakage column survived. Three defenses in code, not just in assumptions.

2. Data source

The **Los Alamos National Laboratory enterprise telemetry dataset** — five gzipped event streams totaling **~11 GB compressed** — capturing **58 days** of activity inside a real corporate network. Each stream tells a different story: `auth` (logon attempts, 7.1 GB), `flows` (network connections, 1.0 GB), `proc` (process start/stop events, 2.2 GB), `dns` (name lookups, 176 MB), and `redteam` (4.7 KB of ground-truth attack labels — only **596 positive (host, hour) windows out of 13.9 M total**, a 0.0043 % positive rate). Raw files land in `s3://lanl-cyber-pipeline/lanl/bronze/` and are never modified, so any downstream artifact can be rebuilt from a known starting point.

3. Pipeline architecture

The whole system runs on AWS, with **three of the four spec-required layers** using distributed or cloud infrastructure (the spec asks for two). The flow is six stages, each producing a concrete artifact: **(1) Bronze on S3** — raw `.txt.gz` preserved as a forensic record. **(2) Bronze → Silver, PySpark on EMR** (`jobs/bronze_to_silver_emr.py`) — applies explicit Spark schemas, casts types, trims strings, adds an `event_day_bucket` partition column, and writes parquet per source. Row grain is preserved (one auth event = one row, etc.). **(3) Silver → Gold, PySpark on EMR** (`jobs/silver_to_gold_emr.py`) — divides relative event time into one-hour windows, aggregates each source by `(computer, time_window)`, joins all five into one wide table, derives ratios (`auth_failure_ratio`, `flows_bytes_per_event`, ...), and builds the future-window target with `lead()`. **The row grain changes here**: gold is now one host-hour per row — the natural unit of "is this host about to be a problem?" **(4) Training + MLflow** (`jobs/train_model.py`) — reads gold from S3, drops leakage columns, fits two scikit-learn pipelines, logs every run to MLflow tracking, registers the winner under `lanl_threat_predictor`, and aliases it `Production`. **(5) Serving — FastAPI on EC2** — seven endpoints, including `/predict`, `/explain` (per-feature log-odds decomposition — exact math, not a SHAP approximation), and `/counterfactual` (smallest single-feature change that would push predicted risk below 20 %). **(6) UI — Streamlit Cloud** — four-tab dashboard with preset scenarios, a Plotly verdict gauge, the explainer chart, the counterfactual table, and live model metrics — all hitting the EC2 API.

4. Model approach

Four scikit-learn pipelines compete head-to-head, each shaped as `ColumnTransformer(SimpleImputer(median) → StandardScaler) → classifier`: a **logistic-regression baseline**, a **balanced random forest**, **XGBoost** (gradient-boosted trees), and **LightGBM** (histogram-based gradient boosting). Every run is logged to MLflow tracking and the validation-F1 winner is registered under `lanl_threat_predictor` aliased `Production`. On this dataset **LightGBM wins** — test ROC AUC = **0.881**, test PR AUC = **0.0390**, **a 35× improvement over the LR baseline's PR AUC of 0.0011** and 4× the per-row precision of the random forest. To keep the `/counterfactual` endpoint functional regardless of which model wins, the LR pipeline is also persisted as a **linear sidecar** that the API loads at startup — exact closed-form counterfactuals stay available even when a tree model serves predictions. The `/explain` endpoint adapts automatically: linear log-odds decomposition for LR, exact TreeSHAP for XGBoost and LightGBM. Inputs are ~43 engineered behavioral features. The split is **stratified random 60/20/20** — *not* time-aware: a strict time-aware split would have placed every one of the 596 positives in training (the red-team campaign ends mid-record), leaving valid/test with zero positives and every metric collapsing to NaN. **The decision threshold is not hard-coded to 0.5** — the API exposes `/threshold_analysis` (the PR curve from validation) and `/cost_optimal_threshold` (returns the threshold that minimizes expected operational cost for a user-supplied FP/FN cost ratio). The UI surfaces both as an interactive cost-matrix calculator.

5. Learnings

First, leakage is easy to miss and humiliating to catch late. The first version of the model had validation F1 near 1.0 — the universal sign that the model is cheating. Fixing it required three layers of defense in code: a future-window target built with `lead()` instead of the current window, dropping every current-window red-team aggregate from features, and an `assert_no_leakage()` runtime check so a future contributor cannot accidentally re-introduce a leak. Metrics dropped to honest numbers, and the predictions started meaning something. **Second, infrastructure pragmatism beats sticking to a plan.** We began on Databricks because it was the path of least resistance from class, but free-tier managed storage couldn't carry an 11 GB raw dataset. We pivoted to **S3 + EMR + EC2** without abandoning the medallion structure, the MLflow story, or the explainability layer. With more time we'd add a **streaming path** (Kinesis → Spark Structured Streaming → live scoring), an **EventBridge-driven retraining schedule** that re-fits the model weekly, **multi-feature counterfactual paths** so the recommender suggests combined interventions instead of single-feature ones, and **calibration analysis** (reliability diagrams + isotonic / Platt scaling) so the predicted probabilities are interpretable as actual frequencies, not just rankings.